Sistema Avanzado de Medición de Reflejos

Raquel Arias García, Diana Vanessa Guerra López, Claudia Elena Durango Cossio
Electrónica digital II

Índice

1. Sistema Avanzado de Medición de Reflejos **2**

2. Resumen **2**

3. Descripción del hardware **2**

4. Descripción del software **4**

5. Estructura del código **5**

6. Explicación de funciones principales **5**

7. Comunicación (si aplica) **7**

8. Interfaz de usuario (si aplica) **7**

9. Procedimiento de prueba **7**

10. Manejo de errores y seguridad **8**

1. Sistema Avanzado de Medición de Reflejos

- **Nombre del proyecto:** Sistema Avanzado de Medición de Reflejos
- **Integrantes del equipo: Raquel Arias García, Diana Vanessa Guerra López, Claudia Elena Durango Cossio**

2. Resumen

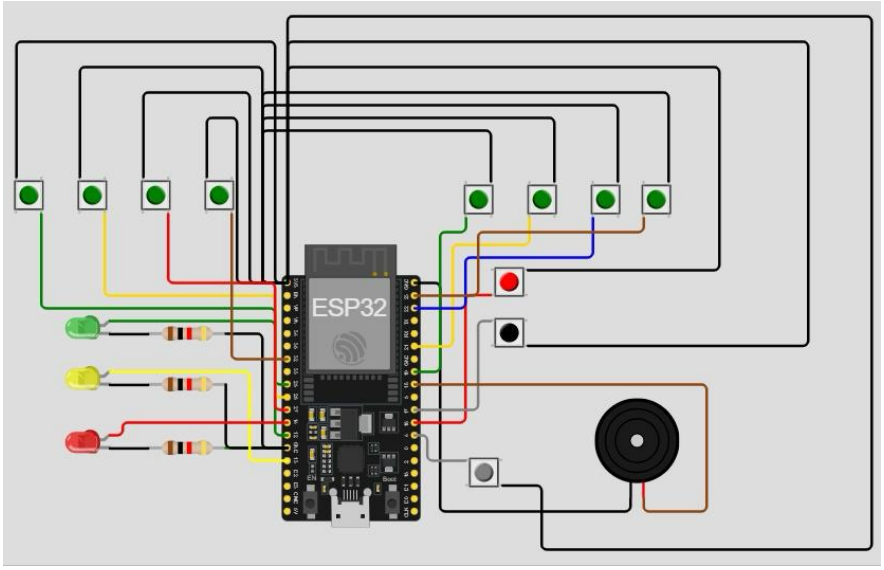
El presente proyecto consiste en el diseño e implementación de un sistema avanzado de medición de reflejos basado en un microcontrolador ESP32. El sistema permite seleccionar modo de uno o dos jugadores mediante un menú interactivo, generar estímulos visuales (LEDs) y sonoros (buzzer) de forma aleatoria, medir el tiempo de reacción en milisegundos y asignar puntuaciones con penalizaciones por errores. Adicionalmente, se implementó un tercer modo de juego activado por interrupción, basado en la dinámica "Simón Dice", el cual incrementa progresivamente la dificultad. El sistema incluye control antirrebote por software, manejo de LEDs mediante registros, sistema de puntajes robusto y visualización clara de resultados por terminal.

3. Descripción del hardware

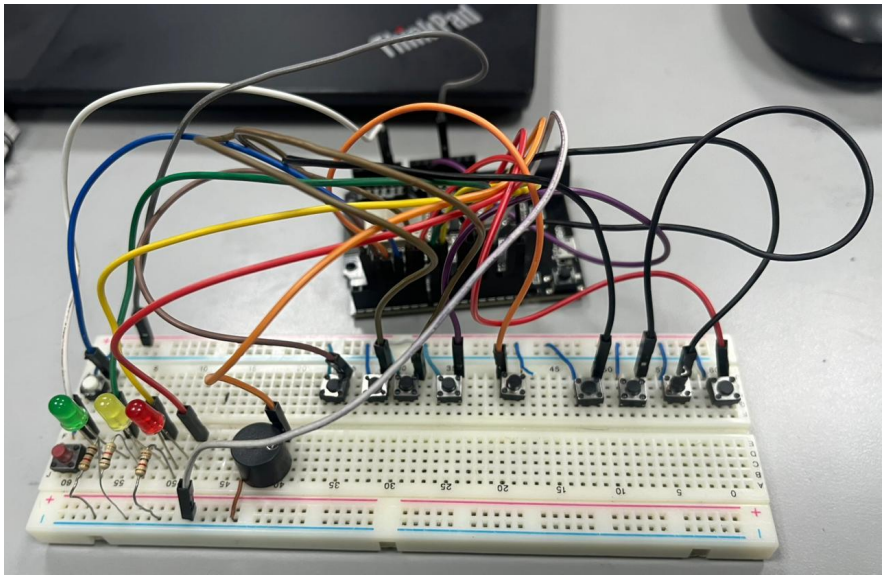
- **ESP32** (modelo con ADC y pines GPIO)
- **LED 1:** Pin 12
- **LED 2:** Pin 13
- **LED 3:** Pin 14
- **Boton J1-1:** Pin 25
- **Boton J1-2:** Pin 26
- **Boton J1-3:** Pin 27
- **Boton J1-4:** Pin 32
- **Boton J2-1:** Pin 19
- **Boton J2-2:** Pin 21
- **Boton J2-3:** Pin 22
- **Boton J2-4:** Pin 23
- **Boton de Inicio:** Pin 16
- **Boton Fin:** Pin 4

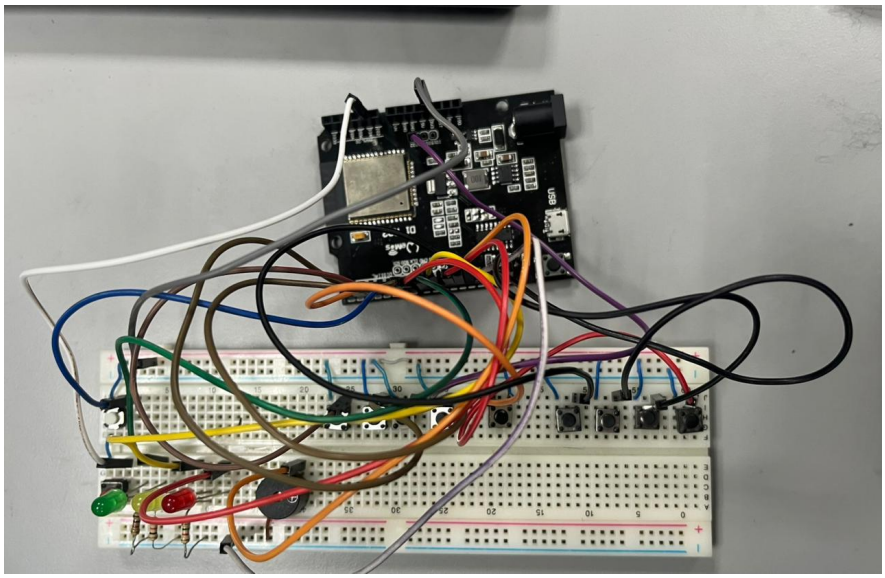
- **Boton Modo Simon:** Pin 17
- **Fuente de alimentación:** 3.3V (USB o regulada)

Diagrama de conexión:



Montaje Fìsico :





4. Descripción del software

- **Lenguaje usado:** MicroPython
- **Librerías:** `machine`, `time`,
- **IDE:** Thonny

Flujo general del programa:

1. Definicioin de pines de entradas y salidas (Leds y Buzzer)
2. Configuración de interrupciones (Modo Simón Dice) y Boton inicico y Fin
3. Mostrar en pantalla el estado del juego
4. Seleccionar número de jugadores
5. Inicio de la ejecución del modo del juego
6. Se muestra en terminal: Mensaje de inicio, Solicitud para seleccionar número de jugadores. Selección del número de jugadores .Se determina si el juego será de uno o dos jugadores utilizando los botones del Jugador 1.
7. Ejecución del modo de juego
8. Modo Reflejos: Se genera un tiempo aleatorio entre 1 y 10 segundos. Se activa un estímulo aleatorio (LED o buzzer). Se mide el tiempo de reacción con time.ticks$_ms$().
8. Se detecta el primer jugador que responde correctamente. Se asigna puntaje (+1si acierta, 1 si se equivoca). Se muestran resultados en la terminal.
9. Modo Simón Dice

10. Se genera una secuencia creciente de estímulos. Se muestran uno a uno. El jugador debe repetir la secuencia. Si falla, termina el modo. Puede finalizar manualmente con el botón FIN.

11. Actualización de puntajes: Después de cada ronda se imprimen: Estímulo generado, Ganador, Tiempo de reacción, Puntajes acumulados

12. Detención del juego: En cualquier momento el botón FIN (GPIO 4) puede detener el juego.

5. Estructura del código

- Archivo principal: `còdigomain.py`
- Se definen variables globales para controlar el estado del sistema:
- puntaje$_j 1, puntaje_j 2, modo_j uego, bandera_s imon.$
- Código lineal dentro de un ciclo infinito con lectura y decisión en tiempo real.
- El programa está estructurado en funciones modulares:
- encender$_p in(pin) y apagar_p in(pin). Controlan LEDs y buzzer$
- mediante acceso directo a registros del ESP32.
- esperar$_b oton(lista_b otones) Leebotones con control de antirrebote.$
- Evita lecturas falsas mediante retardo de 50 ms
- modo$_s imon() Contiene la lógica completa del modo Simón Dice$:
- Generación de secuencia, Visualización, Validación, Finalización automática o manual
- Interrupción activar$_s imon(pin) Función asociada al botón de cambio de modo.$
- Modifica la variable bandera$_s imon cuando se detecta un flanco ascendente.$

6. Explicación de funciones principales

- `encender`$_p in(pin)$: $Enciende el LED o buzzer correspondiente al pin indicado.$

- `La activación se realiza mediante manipulación directa de registros del ESP32 (machine.mem32).`

- Escribiendo en el registro GPIO$_O UT_W 1TS_R EGparacolocarelpinenestadoHIGH(3{,}3V)$.

- esperar$_b oton(lista_b otones) : Esperahastaqueunodelosbotonesdeljugadorseapresionado.$

- Implementa control de antirrebote por software mediante un retardo de 50 ms.
- Verifica nuevamente el estado del botón antes de confirmar la pulsación.
- Devuelve el índice (0 a 3) del botón presionado, permitiendo comparar si corresponde al estímulo generado.
- modo$_s imon() : EjecutaelmododejuegoSimónDice.$
- Genera una secuencia creciente de estímulos aleatorios (LEDs y buzzer)que se almacenan en una lista.
- La secuencia se muestra al jugador activando cada salida durante untiempo determinado.
- El jugador debe repetir la secuencia correctamente; si falla, el modo finaliza.
- Mostrando la ronda Puede detenerse manualmente mediante el botón FIN.
- activar$_s imon(pin)) : Rutinadeinterrupciónasociadaalbotóndecambiodemodo(GPIO17).$

- Se ejecuta cuando ocurre un flanco ascendente (IRQ$_R ISING$).
- activa la variable global bandera$_s imon.$
- Permitiendo que el sistema cambie al modo Simón Dice.
- Esto sin detener el programa principal.
- Bloque principal Juego Reflejos (dentro del while True):
- Este bloque ejecuta la lógica del modo Reflejos: Genera un estímulo aleatorio.
- Activa el LED o buzzer correspondiente. Mide el tiempo de reacción usando time.ticks$_m s()$.
- Determina el primer jugador en responder. Asigna puntaje (+1 por acierto, 1 por error).
- Muestra resultados en la terminal.

7. Comunicación (si aplica)

Comunicación Serial USB (UART integrada): El ESP32 envía información a través del puerto USB hacia la consola del IDE Thonny, donde se muestran: Mensajes de inicio del sistema. Selección del número de jugadores. Número de ronda. Estímulo generado. Tiempo de reacción en milisegundos. Puntajes acumulados.Resultados finales.Información del modo Simón Dice. Esta comunicación permite monitorear el comportamiento del sistema en tiempo real y verificar el correcto funcionamiento del programa.

8. Interfaz de usuario (si aplica)

El sistema no incluye interfaz gráfica, pantalla táctil ni interfaz web. La interacción con el usuario se realiza mediante dos mecanismos principales: Consola Serial. Se utiliza para mostrar: Estado del sistema, Modo seleccionado, Información de cada ronda, Tiempo de reacción, Puntajes acumulados, Puntaje final. Esto proporciona retroalimentación clara y facilita la supervisión del juego. Interfaz Física Está compuesta por:3 LEDs (GPIO 12, 13 y 14) como estímulos visuales,1 buzzer (GPIO 18) como estímulo sonoro,Botones de los jugadores para responder, Botón de inicio (GPIO 16), Botón de fin (GPIO 4), Botón de cambio de modo (GPIO 17). Los jugadores interactúan directamente con los pulsadores, lo que hace que el sistema sea completamente autónomo y funcional sin necesidad de pantalla adicional.

9. Procedimiento de prueba

1. Conectar el ESP32 al computador mediante cable USB.
2. Abrir el IDE Thonny y seleccionar el intérprete MicroPython del ESP32.
3. Verificar que los LEDs estén conectados correctamente a los pines 12, 13 y 14.
4. Verificar que el buzzer esté conectado al pin 18.
5. Comprobar que los botones estén conectados a los pines correspondientes según el diccionario PINES.
6. Ejecutar el programa.
7. Presionar el botón de inicio (GPIO 16).
8. Seleccionar el número de jugadores usando los botones del jugador 1.
9. Observar en la consola serial la información de cada ronda:
10. Estímulo generado.
11. Ganador.
12. Tiempo de reacciòn

13. Puntajes.
14. Presionar el botón de modo (GPIO 17) para activar Simón Dice.
15. Presionar el botón FIN (GPIO 4) para detener el juego.
16. Verificar que el sistema regrese correctamente al estado inicial.

10. Manejo de errores y seguridad

- Se implementó control de antirrebote.
- Por software en la función esperar$_b oton()paraevitarlecturasfalsas.$
- Se controla que el puntaje solo se descuente una vez por error.
- Mediante banderas (error$_j 1, error_j 2$).
- El botón FIN permite salir del juego en cualquier momento.
- El modo Simón puede finalizar manual o automáticamente si el jugador falla.
- Seguridad Eléctrica:
- El ESP32 opera a 3.3 V lógicos.
- Se deben usar resistencias limitadoras en los LEDs.
- No se deben conectar dispositivos que superen 3.3 V directamente a los GPIO.
- El buzzer debe ser compatible con 3.3 V.
- Se recomienda evitar presionar múltiples botones simultáneamente para prevenir lecturas inesperadas.